

Taxi ERP System Design

A zero-budget operations platform for taxi fleet billing, duty management, and accounting.

Covers the full design of a modular ERP spanning Master, Daily Work, Accounts, and Reports panels. Built on self-hosted Supabase and React/Vite, deployable on Oracle Always Free tier with Cloudflare Tunnel for ingress and Backblaze B2 for backups.

STACK	React 18 + Vite 5 · Self-hosted Supabase (PostgreSQL 16, PostgREST, GoTrue, Storage)
HOSTING	Oracle Always Free ARM A1.Flex (4 vCPU / 24 GB) · Cloudflare Tunnel
COST	\$0/month operational · all services on free tiers

Table of Contents

1. Executive Summary	3
2. Tech Stack Validation & Recommendations	3
2.1 Validation Summary	4
2.2 Why self-hosted Supabase instead of Supabase Cloud free tier	4
2.3 Suggested tweaks (do not change architecture)	4
3. System Architecture Overview	5
3.1 Component responsibilities	6
3.2 Data flow: duty slip creation example	6
4. Database Schema Design	7
4.1 core.companies	7
4.2 core.user_profiles	7
4.3 master.customers	8
4.4 master.vehicles and supporting tables	8
4.5 master.rates	10
4.6 master.gst_config	10
4.7 operations.duty_slips	11
4.8 billing.bills and bill_duty_slips	12
4.9 master.document_sequences	13
4.10 accounts.ledger_entries	14
4.11 system.audit_log and system.settings	15
4.12 Row-Level Security policies	16
5. Module Specifications - Master Panel	17
5.1 Company Detail page	17
5.2 Utilities page - Manage Vehicles and Document No. Control	17
5.3 Customer page	17
5.4 Rate Management page	17
5.5 GST Management page	18
6. Module Specifications - Daily Work Panel	18

6.1 Duty Slip page	18
6.2 Flexible duty type - custom rate popup	18
6.3 Billing page	18
6.4 Change / Cancel Bill page	19
6.5 Print Bill or Duty Slip page	19
7. Module Specifications - Accounts Panel	20
7.1 Ledger Book page	20
7.2 Auto-posting rules	20
8. Module Specifications - Reports Panel	20
8.1 Bill Cover report	20
8.2 Bill Register report	21
8.3 Duty Register report	21
9. Cross-Cutting Concerns	21
9.1 Authentication and role model	21
9.2 Audit log mechanism	21
9.3 PDF export pipeline	21
10. Deployment & Free Tier Strategy	22
10.1 VM provisioning checklist	22
10.2 Docker Compose topology	22
10.3 Backup strategy	23
11. Modular Extensibility Plan	23
11.1 Schema namespaces per panel	23
11.2 Dependency direction	23
11.3 Soft references for optional cross-panel links	24
12. Implementation Roadmap	24
12.1 Risk register	24

1. Executive Summary

This document specifies the complete system design for a **Taxi ERP** platform intended for a single-operator taxi business that needs to manage customers, vehicles, duty slips, billing, GST, and accounting in one modular web application. The design prioritizes a zero recurring-cost operating model, ease of maintenance by a single solo developer, and the ability to add or remove functional panels without breaking the rest of the system. Every architectural decision in this document is evaluated against these three constraints so that future changes do not silently compromise the cost, operability, or extensibility goals.

The platform is organized into four top-level panels - **Master**, **Daily Work**, **Accounts**, and **Reports** - mirroring the real-world workflow of a taxi operator. The Master panel holds reference data (company, customers, vehicles, rates, GST configuration) that the Daily Work panel consumes to produce duty slips and bills. The Accounts panel records money in and out, while the Reports panel aggregates the same underlying data into bill covers, registers, and printable outputs. Each panel is treated as an independent feature module with its own database schema namespace, API surface, and UI routes, so it can be removed without leaking its dependencies into other panels.

The recommended stack - **React 18 + Vite 5** on the frontend and **self-hosted Supabase** (PostgreSQL 16 + PostgREST + GoTrue + Storage) on the backend - is validated against the project constraints in Chapter 2. The deployment target is the **Oracle Cloud Always Free tier** (ARM A1.Flex, 4 vCPU / 24 GB RAM), fronted by a Cloudflare Tunnel for free HTTPS ingress and protected by nightly Backblaze B2 backups. The entire stack runs without paid services and stays within free-tier quotas for the foreseeable workload of a single taxi business.

The remainder of this document is structured as follows: Chapter 2 validates the proposed technology stack and recommends minor optimizations. Chapter 3 presents the overall system architecture with a component diagram. Chapter 4 contains the full database schema - tables, columns, types, foreign keys, and row-level security policies. Chapters 5 through 8 specify each of the four functional panels in detail. Chapter 9 covers cross-cutting concerns (authentication, audit, PDF export). Chapter 10 explains the deployment topology on the Oracle Free tier. Chapter 11 documents the modular extensibility strategy. Chapter 12 closes with a phased implementation roadmap.

2. Tech Stack Validation & Recommendations

The user-proposed stack - **self-hosted Supabase on Oracle Free Tier with a React + Vite frontend** - is fundamentally sound and well matched to the constraints of a zero-budget, solo-maintained taxi ERP. Supabase gives a Postgres database, REST API, authentication, and file storage out of the box, eliminating the need to write a custom backend. React + Vite gives a fast SPA experience with a thin client-side bundle that can be hosted from the same VM or even from Cloudflare Pages for free. The full Supabase self-hosted stack runs comfortably in 4-6 GB of RAM on the Oracle ARM instance, leaving plenty of headroom for the frontend build, file storage, and monitoring sidecars.

That said, four minor adjustments make the stack significantly easier to operate over the long term. None of these changes the underlying architecture; they are pragmatic tweaks that lower maintenance burden and protect against free-tier policy changes.

2.1 Validation Summary

Component	Verdict	Notes
PostgreSQL 16 (self-hosted)	Keep	Single source of truth. Use logical backups + WAL archiving.
PostgREST (via Supabase)	Keep	Auto-CRUD over the schema is sufficient - no custom API layer needed.
GoTrue (Supabase Auth)	Keep	Email/password and magic-link login. Role claims via JWT.
Supabase Storage API	Keep	S3-compatible; on-disk storage on Oracle VM.
React 18 + Vite 5	Keep	Vite 5 has fastest dev/build cycle for a solo dev.
Cloudflare Tunnel	Add	Replaces need for inbound firewall/ports - critical on Oracle Free.
Backblaze B2 (10GB free)	Add	Off-VM nightly encrypted backups. B2 is cheaper than S3 free tier expiration.
Coolify / Dokploy	Add	Optional one-click Docker deploy UI. Saves writing raw Compose for non-experts.
Uptime Kuma	Add	Free self-hosted monitoring. Pushes webhook alerts on failures.

Table 2.1 - Stack component verdicts and minor adjustments.

2.2 Why self-hosted Supabase instead of Supabase Cloud free tier

Supabase Cloud offers a free tier (500 MB DB, 1 GB storage, 50,000 monthly active users) that is technically sufficient for a small taxi business. However, the project pauses the database after seven days of inactivity, caps the database at 500 MB which a multi-year billing history will eventually exceed, and exposes no in-region point-in-time recovery. Self-hosting on Oracle Free removes these caps: the VM gives 24 GB RAM and 200 GB of block storage that comfortably holds years of duty slips and bills, and the operator controls backup retention directly. The trade-off is operational responsibility, which is mitigated by the automated backup and monitoring stack described in Chapter 10.

2.3 Suggested tweaks (do not change architecture)

- **Use a Docker Compose pinned-version Supabase stack** rather than cloning the master branch. Pinning avoids surprise breaking changes when Supabase cuts a new release. Track the *self-hosted* branch instead of *main* - it is more stable for self-hosters.
- **Add Cloudflare Tunnel as the sole ingress path.** Oracle Free Tier instances are often probed by bots; closing all inbound ports and exposing the app only through a Cloudflare Tunnel eliminates an entire class of attack surface and gives free TLS.

- **Enable pg_cron + pg_net extensions** for scheduled jobs (nightly backup, GST period reminders, ledger auto-reconciliation). Removes the need for a separate cron container.
- **Use the Vite dev server in production-like staging** via vite preview for the simplest possible deploy, or build to a static dist/ folder and serve via Caddy/nginx. The SPA has no SSR needs.
- **Store PDFs as computed on the client** using @react-pdf/renderer. Bills and duty slips are rendered in the browser; no server-side headless Chrome is needed. This saves ~300 MB of RAM that a headless Chrome service would otherwise consume.

Stack verdict

The proposed stack is approved. All four tweaks above are additive and reversible. No component needs to be replaced.

3. System Architecture Overview

The Taxi ERP is a classic **single-page application + BaaS** architecture: a static React SPA in the browser talks over HTTPS to a self-hosted Supabase backend running on a single Oracle Free Tier ARM instance. The diagram below shows the four logical layers - browser, Cloudflare edge, application/DB VM, and external services - and the free-tier role each component plays. All network traffic flows out through the Cloudflare Tunnel; the VM never exposes a public inbound port, which dramatically reduces the attack surface and removes the need to manage TLS certificates manually.

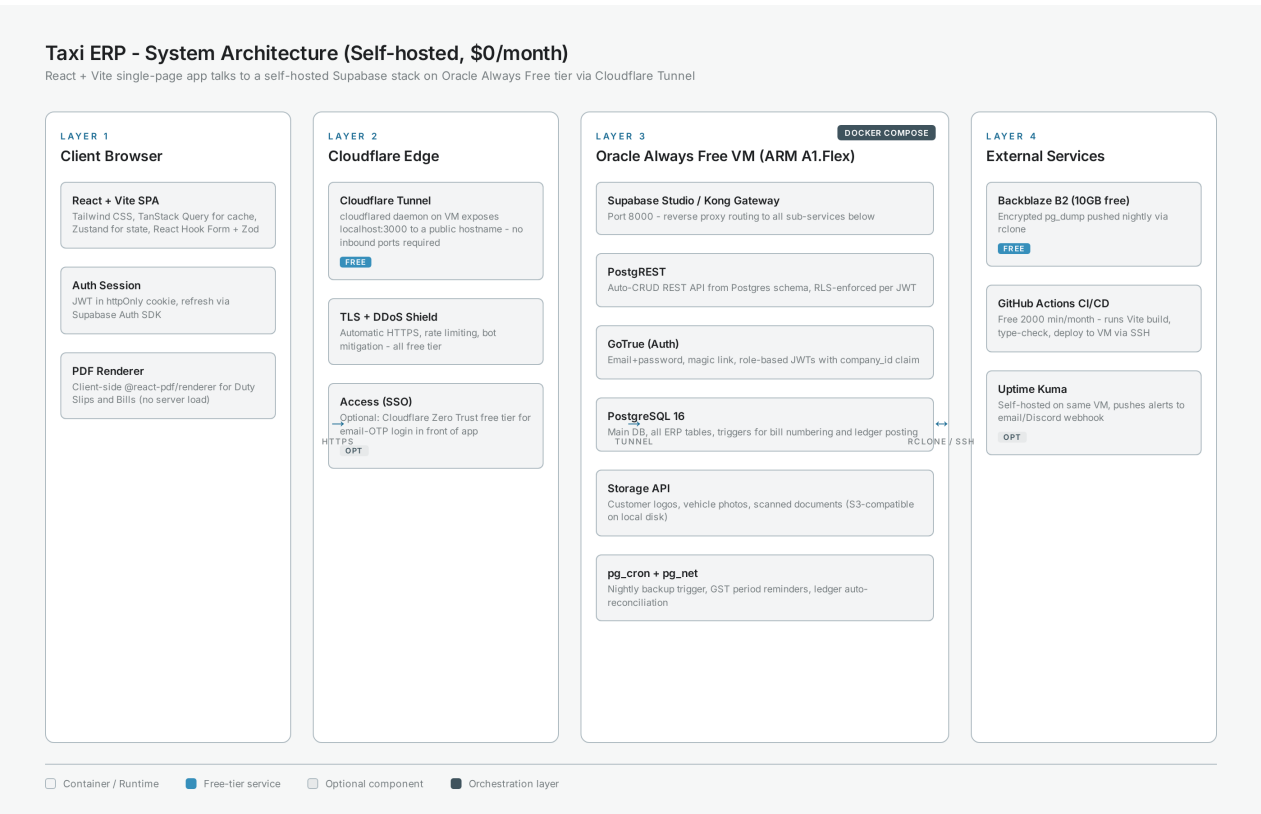


Figure 3.1 - End-to-end architecture. All services run on free tiers; no paid component is required.

3.1 Component responsibilities

Layer 1 - Client Browser. The React SPA is the only user-facing surface. It is built with Vite to a static dist/ folder and served either from the same Oracle VM (via Caddy) or from Cloudflare Pages. The SPA holds TanStack Query for server-state caching (duty slips, bills, customers), Zustand for ephemeral UI state (modal open/close, form drafts), and React Hook Form + Zod for form validation. PDF generation for duty slips and bills happens entirely in the browser via @react-pdf/renderer - no server round-trip is needed for printing.

Layer 2 - Cloudflare Edge. The Cloudflare Tunnel daemon (cloudflared) runs as a systemd service on the VM. It opens an outbound persistent connection to Cloudflare and registers a public hostname (e.g. taxi.example.com) that maps to localhost:8000 on the VM. Cloudflare terminates TLS, applies rate limiting and bot mitigation, and forwards traffic to the tunnel. Optional Cloudflare Zero Trust (free for up to 50 users) adds email-OTP SSO in front of the app, useful for restricting access to a small number of operators.

Layer 3 - Oracle Free Tier VM (Docker Compose). The application VM runs the full self-hosted Supabase stack inside Docker containers managed by a single docker-compose.yml. Kong (or the newer Supabase API gateway) acts as the public entry on port 8000 and routes traffic to PostgREST (auto-CRUD REST API over Postgres), GoTrue (JWT issuance and refresh), Storage API (S3-compatible file storage on local disk), and PostgreSQL 16 itself. The pg_cron and pg_net Postgres extensions run scheduled SQL jobs - nightly backup triggers, GST reminder emails, and ledger reconciliation - without an external cron container.

Layer 4 - External services. Three external services round out the operational story: Backblaze B2 (10 GB free, ~\$0.005/GB beyond) stores nightly encrypted pg_dump archives pushed via rclone; GitHub Actions (free 2,000 minutes/month for private repos) runs the Vite build, type-check, and SSH-based deploy to the VM; and Uptime Kuma (self-hosted in a small container on the same VM) pings the public hostname every 60 seconds and pushes alerts to Discord or email on failure. None of these services incur a recurring fee at the expected workload.

3.2 Data flow: duty slip creation example

To make the architecture concrete, consider a duty slip creation flow. The operator opens the Daily Work panel in the browser, selects the customer and vehicle, and types duty start/end times. The SPA validates the form locally (Zod), then issues a POST to POST /rest/v1/duty_slips on PostgREST. PostgREST verifies the JWT, enforces the row-level security policy (operator can only insert into rows where duty_slips.company_id = JWT.company_id), fires the before insert trigger to fetch the next duty slip number from the document_sequences table, and inserts the row. The same transaction also writes an entry into audit_log. PostgREST returns the inserted row to the SPA, which updates the TanStack Query cache. The operator then clicks *Print*, which renders the duty slip as a PDF entirely in the browser - no server round-trip, no headless Chrome process.

4. Database Schema Design

The schema is organized into six logical namespaces - **core**, **master**, **operations**, **billing**, **accounts**, and **system**. Each namespace maps to a Postgres schema (literally, CREATE SCHEMA master) so that tables can be added or removed panel-by-panel without colliding with other panels. All tables use bigserial primary keys, created_at/updated_at timestamps, and a company_id foreign key for multi-tenant isolation. Row-level security policies enforce that users can only see rows belonging to their own company.

Conventions

PK = primary key, FK = foreign key, NN = not null, UQ = unique, IDX = index. All id columns are bigserial. All company_id columns have an RLS policy; examples shown in section 4.7.

4.1 core.companies

Holds the operator company profile. There is typically exactly one row per deployment, but the schema supports multiple companies so that a single VM can serve sister concerns if needed later. The company_id from this table is the tenant boundary for every other table.

```
CREATE TABLE core.companies (
  id          bigserial PRIMARY KEY,
  name        text NOT NULL,
  legal_name  text,                -- for GST invoice
  owner_name  text,
  gstin       text UNIQUE,         -- 15-char GST identification number
  pan         text,
  address_line1 text NOT NULL,
  address_line2 text,
  city        text NOT NULL,
  state       text NOT NULL,      -- e.g. 'Delhi' - drives IGST
  vs CGST/SGST
  pincode     text NOT NULL,
  phone       text,
  email       text,
  logo_path   text,               -- storage bucket key
  is_active   boolean DEFAULT true,
  created_at  timestampz DEFAULT now(),
  updated_at  timestampz DEFAULT now()
);
```

Column	Type	Notes
id	bigserial	PK; tenant boundary
name	text	Display name shown in UI
gstin	text	15-char GSTIN; uniqueness enforced
state	text	Drives IGST/CGST+SGST logic in gst_config

Table 4.1 - core.companies key columns (full DDL above).

4.2 core.user_profiles

Supabase Auth provides the `auth.users` table for credentials. A 1:1 profile table in core holds application-specific metadata: the company the user belongs to, and their role. The role enum drives UI permission checks; RLS policies use the same value to gate write access.

```
CREATE TYPE user_role AS ENUM ('owner', 'operator', 'accountant', 'viewer');

CREATE TABLE core.user_profiles (
  id          uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  full_name   text NOT NULL,
  role        user_role NOT NULL DEFAULT 'viewer',
  is_active   boolean DEFAULT true,
  created_at  timestampz DEFAULT now(),
  updated_at  timestampz DEFAULT now()
);

-- Auto-create profile on auth.users insert via trigger
-- JWT claim: auth.jwt() -> 'user_role' is populated by a Supabase hook.
```

4.3 master.customers

The customer table stores both business clients (companies that book cars regularly on credit) and one-time personal clients (walk-in cash customers). The `client_type` enum differentiates the two; the rate-management and GST-management screens filter by this field. For one-time clients, the booking record itself stores the name and contact instead of creating a customer row.

```
CREATE TYPE client_type AS ENUM ('company', 'personal');

CREATE TABLE master.customers (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  client_type client_type NOT NULL DEFAULT 'company',
  name        text NOT NULL, -- contact person for company;
  company_name text, -- customer's company (only for personal)
  client_type client_type NOT NULL DEFAULT 'company',
  gstin       text, -- required for B2B billing; NULL for personal
  address_line1 text,
  address_line2 text,
  city         text,
  state        text, -- drives gst_config lookup
  pincode      text,
  phone        text NOT NULL,
  email        text,
  pan          text,
  is_active    boolean DEFAULT true,
  notes        text,
  created_at   timestampz DEFAULT now(),
  updated_at   timestampz DEFAULT now()
);

CREATE INDEX idx_customers_company_type ON master.customers(company_id, client_type);
CREATE INDEX idx_customers_name ON master.customers(name);
```

4.4 master.vehicles and supporting tables

Vehicles are classified by two orthogonal taxonomies - **vehicle_group** (e.g. sedan, SUV, tempo) and **vehicle_type** (e.g. AC, non-AC, electric). Both taxonomies are user-defined lookup tables so the operator can add new groups without code changes. The vehicle table references both, plus the company for tenancy.

```
CREATE TABLE master.vehicle_groups (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  name        text NOT NULL,          -- e.g. "Sedan";
  description text,
  display_order int DEFAULT 0,
  UNIQUE(company_id, name)
);

CREATE TABLE master.vehicle_types (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  name        text NOT NULL,          -- e.g. "AC";, "Non-
AC";, "Electric";
  UNIQUE(company_id, name)
);

CREATE TABLE master.vehicles (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  registration_no text NOT NULL,      -- e.g. "DL 01 AB 1234";
  vehicle_group_id bigint REFERENCES master.vehicle_groups(id),
  vehicle_type_id bigint REFERENCES master.vehicle_types(id),
  make        text,
  model       text,
  year        int,
  color       text,
  chassis_no  text,
  engine_no   text,
  rc_expiry   date,
  insurance_no text,
  insurance_expiry date,
  permit_no   text,
  permit_expiry date,
  is_active   boolean DEFAULT true,
  notes       text,
  created_at  timestampz DEFAULT now(),
  updated_at  timestampz DEFAULT now(),
  UNIQUE(company_id, registration_no)
);

CREATE INDEX idx_vehicles_group ON master.vehicles(company_id, vehicle_group_id);
```

Table	Purpose	Cardinality
master.vehicle_groups	Sedan / SUV / Tempo	1 row per group per company
master.vehicle_types	AC / Non-AC / Electric	1 row per type per company
master.vehicles	The actual cars	1 row per vehicle per company
master.vehicles.vehicle_group_id	FK to vehicle_groups	Many vehicles per group
master.vehicles.vehicle_type_id	FK to vehicle_types	Many vehicles per type

Table 4.2 - Vehicle taxonomy relationships.

4.5 master.rates

The rates table is the heart of the billing engine. Each row pins a (customer, vehicle, duty_type) triple to a rate card. **Duty type** is one of: per_km (charge by kilometers), per_hour (charge by hours), per_day (fixed daily rate), local_package (fixed for a city package), outstation (out-of-city fixed or per-km), and flexible (no rate applied - operator enters a custom amount on each duty slip).

```
CREATE TYPE duty_type AS ENUM (
    &#x27;per_km&#x27;; &#x27;per_hour&#x27;; &#x27;per_day&#x27;;
    &#x27;local_package&#x27;; &#x27;outstation&#x27;; &#x27;flexible&#x27;
);

CREATE TABLE master.rates (
    id                bigint PRIMARY KEY,
    company_id        bigint NOT NULL REFERENCES core.companies(id),
    customer_id        bigint NOT NULL REFERENCES master.customers(id),
    vehicle_group_id   bigint REFERENCES master.vehicle_groups(id),
    vehicle_type_id    bigint REFERENCES master.vehicle_types(id),
    duty_type          duty_type NOT NULL,
    base_rate          numeric(10,2) NOT NULL,          -- base amount
    per_km_rate        numeric(10,2),                  -- charged for per_km / outstation
    per_hour_rate       numeric(10,2),                  -- charged for per_hour
    per_day_rate        numeric(10,2),                  -- charged for per_day /
    local_package
    extra_hour_rate     numeric(10,2),                  -- beyond included hours
    extra_km_rate       numeric(10,2),                  -- beyond included kms
    night_halt_rate     numeric(10,2),                  -- overnight halt charge
    driver_allowance    numeric(10,2),                  -- per-day driver bata
    min_charge         numeric(10,2),                  -- minimum billable amount
    effective_from      date NOT NULL DEFAULT CURRENT_DATE,
    effective_to        date,                          -- NULL = currently active
    is_active           boolean DEFAULT true,
    notes              text,
    created_at          timestamptz DEFAULT now(),
    updated_at          timestamptz DEFAULT now(),
    UNIQUE(company_id, customer_id, vehicle_group_id, vehicle_type_id, duty_type,
           effective_from)
);

CREATE INDEX idx_rates_customer_vehicle ON master.rates(company_id, customer_id,
    vehicle_group_id, vehicle_type_id, duty_type);
```

4.6 master.gst_config

GST calculation rule: if the customer state differs from the operator company state, IGST applies on (base + extra). If the states match, CGST and SGST split the tax equally (each = IGST/2). The gst_config table stores the current tax rates per state pair, allowing the operator to update rates on the GST Management page without code changes. The trigger function fn_calculate_gst() on the bills table reads this config to populate the cgst_amount, sgst_amount, and igst_amount columns on insert.

```

CREATE TABLE master.gst_config (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  customer_id bigint NOT NULL REFERENCES master.customers(id),
  is_interstate boolean NOT NULL,          -- true if customer.state != company.state
  igst_rate   numeric(5,2),                -- e.g. 5.00 = 5%
  cgst_rate   numeric(5,2),                -- half of igst_rate typically
  sgst_rate   numeric(5,2),                -- half of igst_rate typically
  effective_from date NOT NULL DEFAULT CURRENT_DATE,
  effective_to  date,
  is_active     boolean DEFAULT true,
  created_at    timestampz DEFAULT now(),
  updated_at    timestampz DEFAULT now(),
  UNIQUE(company_id, customer_id, effective_from)
);

-- Trigger: derive is_interstate automatically
CREATE OR REPLACE FUNCTION master.fn_set_interstate()
RETURNS TRIGGER AS $$
DECLARE
  customer_state text;
  company_state  text;
BEGIN
  SELECT state INTO customer_state FROM master.customers WHERE id = NEW.customer_id;
  SELECT state INTO company_state  FROM core.companies  WHERE id = NEW.company_id;
  NEW.is_interstate := (customer_state IS DISTINCT FROM company_state);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_gst_interstate BEFORE INSERT OR UPDATE ON master.gst_config
FOR EACH ROW EXECUTE FUNCTION master.fn_set_interstate();

```

GST calculation logic

If is_interstate = true: apply igst_rate to (base_amount + extra_amount). cgst_amount = 0, sgst_amount = 0. **If is_interstate = false:** split equally - cgst_amount = sgst_amount = (base + extra) * (igst_rate / 2) / 100. igst_amount = 0.

4.7 operations.duty_slips

The duty slip is the atomic unit of operational data. Each duty slip captures one assignment: who booked, which vehicle, when the duty started and ended, what rate was applied, and what extra charges were incurred. Bills aggregate one or more duty slips. The document_sequences table controls the visible duty slip number via a before insert trigger - the operator can switch between auto-incrementing sequence and manual entry on the Utilities page.

```

CREATE TABLE operations.duty_slips (
  id                bigserial PRIMARY KEY,
  company_id        bigint NOT NULL REFERENCES core.companies(id),
  duty_slip_no       text NOT NULL,          -- display number, from
document_sequences
  customer_id        bigint NOT NULL REFERENCES master.customers(id),
  vehicle_id         bigint NOT NULL REFERENCES master.vehicles(id),
  rate_id            bigint REFERENCES master.rates(id), -- NULL for
duty_type=&#x27;flexible&#x27;;
  duty_type          duty_type NOT NULL,
  booking_date       date NOT NULL,
  booking_ref        text,                  -- customer&#x27;s PO/booking number
  guest_name         text,
  guest_phone        text,
  pickup_location    text,
  drop_location      text,
  duty_start_dt      timestampz NOT NULL,
  duty_end_dt        timestampz,
  closing_dt         timestampz,          -- when vehicle returned
  opening_km         numeric(10,2),
  closing_km         numeric(10,2),
  total_km           numeric(10,2) GENERATED ALWAYS AS (closing_km - opening_km)
STORED,
  total_hours        numeric(10,2),        -- computed at save time
  base_amount        numeric(12,2) NOT NULL DEFAULT 0,
  extra_km_amount    numeric(12,2) DEFAULT 0,
  extra_hour_amount  numeric(12,2) DEFAULT 0,
  night_halt_amount  numeric(12,2) DEFAULT 0,
  driver_allowance   numeric(12,2) DEFAULT 0,
  other_charges      numeric(12,2) DEFAULT 0,
  other_charges_remarks text,
  total_amount       numeric(12,2) NOT NULL, -- sum of the above
  custom_rate        numeric(12,2),        -- only for duty_type=&#x27;
flexible&#x27;;
  custom_rate_remarks text,
  driver_name        text,
  driver_phone       text,
  status             text NOT NULL DEFAULT &#x27;open&#x27;;, -- open / closed /
billed / cancelled
  bill_id            bigint,                -- set when added to a bill (FK
added in billing.bills)
  created_by         uuid REFERENCES core.user_profiles(id),
  created_at         timestampz DEFAULT now(),
  updated_at         timestampz DEFAULT now(),
  UNIQUE(company_id, duty_slip_no)
);
CREATE INDEX idx_duty_slips_customer ON operations.duty_slips(company_id, customer_id);
CREATE INDEX idx_duty_slips_vehicle ON operations.duty_slips(company_id, vehicle_id);
CREATE INDEX idx_duty_slips_status ON operations.duty_slips(company_id, status,
booking_date);

```

4.8 billing.bills and bill_duty_slips

A bill is a numbered invoice covering one or more duty slips for a single customer. The bill_duty_slips junction table is many-to-many - a duty slip can only belong to one bill at a time (enforced by a partial unique index on duty_slips.bill_id), but historical bills preserve the link even after a duty slip is re-billed following a cancellation. The total amounts on the bill are denormalized into the bills table itself for fast report queries, and recomputed by triggers whenever a linked duty slip is added, removed, or modified.

```

CREATE TABLE billing.bills (
  id                bigserial PRIMARY KEY,
  company_id        bigint NOT NULL REFERENCES core.companies(id),
  bill_no           text NOT NULL,                -- from document_sequences (
sequence_key=&#x27;bill&#x27;);
  bill_date         date NOT NULL DEFAULT CURRENT_DATE,
  customer_id       bigint NOT NULL REFERENCES master.customers(id),
  gst_config_id     bigint REFERENCES master.gst_config(id),
  base_amount       numeric(12,2) NOT NULL DEFAULT 0,    -- sum of duty_slips.base
  extra_amount      numeric(12,2) NOT NULL DEFAULT 0,    -- sum of extra_*
  total_before_tax  numeric(12,2) NOT NULL DEFAULT 0,
  cgst_amount       numeric(12,2) DEFAULT 0,
  sgst_amount       numeric(12,2) DEFAULT 0,
  igst_amount       numeric(12,2) DEFAULT 0,
  total_tax         numeric(12,2) GENERATED ALWAYS AS (cgst_amount + sgst_amount +
igst_amount) STORED,
  total_after_tax   numeric(12,2) GENERATED ALWAYS AS (total_before_tax + cgst_amount
+ sgst_amount + igst_amount) STORED,
  round_off         numeric(12,2) DEFAULT 0,
  grand_total       numeric(12,2) NOT NULL,
  remarks           text,
  status            text NOT NULL DEFAULT &#x27;draft&#x27;; -- draft / issued /
paid / cancelled / adjusted
  cancelled_at      timestamptz,
  cancelled_by      uuid REFERENCES core.user_profiles(id),
  cancel_reason     text,
  created_by        uuid REFERENCES core.user_profiles(id),
  created_at        timestamptz DEFAULT now(),
  updated_at        timestamptz DEFAULT now(),
  UNIQUE(company_id, bill_no)
);
CREATE INDEX idx_bills_customer_date ON billing.bills(company_id, customer_id,
bill_date);
CREATE INDEX idx_bills_status      ON billing.bills(company_id, status);

CREATE TABLE billing.bill_duty_slips (
  bill_id          bigint NOT NULL REFERENCES billing.bills(id) ON DELETE CASCADE,
  duty_slip_id     bigint NOT NULL REFERENCES operations.duty_slips(id) ON DELETE
RESTRICT,
  included_base    numeric(12,2),                -- snapshot at inclusion time
  included_extra   numeric(12,2),
  included_total   numeric(12,2),
  PRIMARY KEY (bill_id, duty_slip_id)
);

-- Partial unique index: a duty slip can be on at most one non-cancelled bill
CREATE UNIQUE INDEX uq_duty_slip_active_bill
ON operations.duty_slips(bill_id)
WHERE bill_id IS NOT NULL AND status &lt;&gt; &#x27;cancelled&#x27;;

```

4.9 master.document_sequences

Controls the visible numbering of duty slips, bills, and any other document types added later. The mode field selects between auto (sequence incremented by trigger) and manual (operator types the number; trigger validates uniqueness only). The next_value column holds the next auto-sequence value; manual mode ignores it.

```

CREATE TYPE sequence_mode AS ENUM (&#x27;auto&#x27;;, &#x27;manual&#x27;);

CREATE TABLE master.document_sequences (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  sequence_key text NOT NULL,          -- &#x27;duty_slip&#x27; | &#x27;
bill&#x27; | future types
  prefix      text,                  -- e.g. &quot;DS-&quot; / &quot;BILL-
&quot;;
  suffix      text,
  next_value  integer NOT NULL DEFAULT 1,
  mode        sequence_mode NOT NULL DEFAULT &#x27;auto&#x27;;,
  padding_length integer NOT NULL DEFAULT 4,  -- zero-pad width
  is_active   boolean DEFAULT true,
  created_at  timestamptz DEFAULT now(),
  updated_at  timestamptz DEFAULT now(),
  UNIQUE(company_id, sequence_key)
);

-- Trigger: assign duty_slip_no
CREATE OR REPLACE FUNCTION operations.fn_assign_duty_slip_no()
RETURNS TRIGGER AS $$
DECLARE
  seq master.document_sequences%ROWTYPE;
  formatted_no text;
BEGIN
  IF NEW.duty_slip_no IS NOT NULL THEN
    RETURN NEW;          -- manual mode: caller provided the number
  END IF;
  SELECT * INTO seq FROM master.document_sequences
    WHERE company_id = NEW.company_id AND sequence_key = &#x27;duty_slip&#x27;;
  FOR UPDATE;
  IF seq IS NULL THEN
    NEW.duty_slip_no := &#x27;DS-&#x27; || NEW.id;
    RETURN NEW;
  END IF;
  formatted_no := COALESCE(seq.prefix,&#x27;&#x27;) || lpad(seq.next_value::text,
    seq.padding_length, &#x27;0&#x27;) || COALESCE(seq.suffix,&#x27;&#x27;);
  NEW.duty_slip_no := formatted_no;
  UPDATE master.document_sequences SET next_value = next_value + 1
    WHERE id = seq.id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_duty_slip_no BEFORE INSERT ON operations.duty_slips
FOR EACH ROW EXECUTE FUNCTION operations.fn_assign_duty_slip_no();

```

4.10 accounts.ledger_entries

Every financial movement - sale (bill issued), receipt (money received from customer), payment (money paid out for fuel, driver bata, maintenance) - is a single row in ledger_entries. The entry_type enum classifies the direction; the linked_bill_id and linked_duty_slip_id columns preserve the traceability back to operational data. The reporting views in Chapter 8 are all built on top of this table.

```

CREATE TYPE ledger_entry_type AS ENUM (
    &#x27;sale&#x27;;,      -- credit sale when bill is issued (debit customer)
    &#x27;receipt&#x27;;,    -- money received from customer (credit customer)
    &#x27;payment&#x27;;,     -- money paid to vendor / driver (debit expense)
    &#x27;adjustment&#x27;;,  -- manual adjustment by accountant
    &#x27;opening_balance&#x27;;
);

CREATE TABLE accounts.ledger_entries (
    id                bigserial PRIMARY KEY,
    company_id        bigint NOT NULL REFERENCES core.companies(id),
    entry_date        date NOT NULL DEFAULT CURRENT_DATE,
    entry_type        ledger_entry_type NOT NULL,
    customer_id        bigint REFERENCES master.customers(id),
    linked_bill_id     bigint REFERENCES billing.bills(id),
    linked_duty_slip_id bigint REFERENCES operations.duty_slips(id),
    debit_amount       numeric(12,2) DEFAULT 0,    -- increase in receivable / expense
    credit_amount      numeric(12,2) DEFAULT 0,    -- decrease in receivable /
increase in income
    narration          text NOT NULL,
    payment_mode        text,                      -- cash / cheque / upi / bank
    reference_no        text,
    created_by          uuid REFERENCES core.user_profiles(id),
    created_at          timestamptz DEFAULT now(),
    updated_at          timestamptz DEFAULT now()
);
CREATE INDEX idx_ledger_company_date ON accounts.ledger_entries(company_id, entry_date);
CREATE INDEX idx_ledger_customer     ON accounts.ledger_entries(company_id,
customer_id);
CREATE INDEX idx_ledger_bill         ON accounts.ledger_entries(company_id,
linked_bill_id);

```

4.11 system.audit_log and system.settings

The audit_log table captures every INSERT / UPDATE / DELETE on the business tables via a generic trigger function. Old and new row snapshots are stored as JSONB so the audit history is self-describing even if the underlying table schema evolves. system.settings holds per-company key/value configuration not significant enough to warrant a column on core.companies - things like default tax rate, preferred PDF font, etc.

```

CREATE TABLE system.audit_log (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL,
  table_name  text NOT NULL,
  record_id   bigint NOT NULL,
  action      text NOT NULL,          -- INSERT / UPDATE / DELETE
  old_row     jsonb,
  new_row     jsonb,
  changed_by  uuid REFERENCES core.user_profiles(id),
  changed_at  timestampz DEFAULT now()
);
CREATE INDEX idx_audit_company_table ON system.audit_log(company_id, table_name,
changed_at);

CREATE TABLE system.settings (
  id          bigserial PRIMARY KEY,
  company_id  bigint NOT NULL REFERENCES core.companies(id),
  setting_key  text NOT NULL,
  setting_value text,
  data_type   text DEFAULT &#x27;string&#x27;;      -- string | number |
boolean | json
  updated_at  timestampz DEFAULT now(),
  UNIQUE(company_id, setting_key)
);

```

4.12 Row-Level Security policies

Supabase/Auth JWT carries the `company_id` and `user_role` claims via a custom auth hook. RLS policies on every tenant-scoped table enforce that a user can only see rows of their own company. The pattern below is the canonical policy - it is replicated on every tenant-scoped table with the same shape.

```

-- Helper SQL function: read company_id from JWT
CREATE OR REPLACE FUNCTION public.current_company_id()
RETURNS bigint LANGUAGE sql STABLE AS $$
  SELECT NULLIF((auth.jwt() ->&#x27;company_id&#x27;), &#x27;&#x27;)::bigint;
$$;

-- Helper SQL function: read user_role from JWT
CREATE OR REPLACE FUNCTION public.current_user_role()
RETURNS text LANGUAGE sql STABLE AS $$
  SELECT COALESCE(auth.jwt() ->&#x27;user_role&#x27;, &#x27;viewer&#x27;);
$$;

-- Example policy: customers table
ALTER TABLE master.customers ENABLE ROW LEVEL SECURITY;

CREATE POLICY customers_tenant_isolation ON master.customers
  FOR ALL
  USING (company_id = public.current_company_id())
  WITH CHECK (company_id = public.current_company_id());

CREATE POLICY customers_write_requires_operator ON master.customers
  FOR INSERT
  WITH CHECK (public.current_user_role() IN (&#x27;owner&#x27;, &#x27;operator&#x27;));

CREATE POLICY customers_update_requires_operator ON master.customers
  FOR UPDATE
  USING (public.current_user_role() IN (&#x27;owner&#x27;, &#x27;operator&#x27;, &#x27;
accountant&#x27;));

```

RLS deployment note

Apply the same three-policy shape (isolation, write-requires-operator, update-requires-operator-or-accountant) to every tenant-scoped table. Use a SQL script generator (a `generate_rls.sql` file in the migrations folder) to apply them in bulk rather than hand-writing each one.

5. Module Specifications - Master Panel

The Master panel is the configuration backbone of the system. Every record created here is consumed by the Daily Work, Accounts, and Reports panels. The five pages described below - Company Detail, Utilities, Customer, Rate Management, and GST Management - cover all reference data needed to operate the rest of the ERP. None of the Master panel pages generate transactional data; they only create and maintain the lookup tables that the other panels join against.

5.1 Company Detail page

A single-record form bound to `core.companies`. The operator opens this page once at first boot to enter their own company name, owner name, GSTIN, address (with state - critical for GST logic), phone, email, and an optional logo upload. After the initial setup, this page is revisited only when contact details change. The logo upload goes to Supabase Storage under `company-logos/` bucket and is referenced by `logo_path`; the path is returned to the SPA as a signed URL on every load.

5.2 Utilities page - Manage Vehicles and Document No. Control

The Utilities page has two tabs. **Tab 1 - Manage Vehicles** provides a CRUD list of all vehicles with filters by `vehicle_group` and `vehicle_type`, plus a quick add form that creates a vehicle with all RC, insurance, and permit expiry dates. The operator can mark a vehicle as inactive (which removes it from new duty-slip dropdowns) without deleting historical records. **Tab 2 - Document No. Control** manages the `document_sequences` table: each row controls the prefix, suffix, next number, mode (auto vs manual), and padding length for one document type. The operator can switch duty slips from auto to manual numbering for example, or reset the next number at the start of a fiscal year.

5.3 Customer page

Stores both **company** clients (B2B - regular bookings on credit) and **personal** clients (one-time cash customers). The `client_type` radio at the top of the form toggles which fields are required: company clients require `company_name` and `gstn` (for B2B invoices); personal clients skip both. State is captured for every customer because it feeds the GST config lookup. The customer list page supports search by name, phone, or GSTIN, and filters by `client_type` and active flag.

5.4 Rate Management page

The rate matrix screen. The operator picks a customer from the dropdown, optionally filters by `vehicle_group/vehicle_type`, and the page renders a table where each row is a (`vehicle_group`, `vehicle_type`, `duty_type`) triple and each editable cell is one of the rate fields (`base`, `per_km`, `per_hour`, `per_day`, `extra_hour`,

extra_km, night_halt, driver_allowance, min_charge). The page uses effective_from/effective_to to time-travel rates - changing a rate inserts a new row with effective_from = today and closes the previous row with effective_to = yesterday, preserving historical billing accuracy. The flexible duty type is special - it has no rate fields, and duty slips of this type skip the rate lookup entirely.

5.5 GST Management page

A per-customer GST configuration screen. The operator picks a customer, the page shows the current IGST rate (interstate) and the derived CGST/SGST rates (typically half each, but editable). The is_interstate flag is computed automatically by the trigger function master.fn_set_interstate() by comparing the customer state to the company state - the operator never sets it manually. The bottom of the page shows a read-only summary of the current rate spread: "Customer is in {state}, your company is in {state}, therefore {IGST | CGST+SGST} applies."

GST decision tree (bill trigger)

On bill insert: read gst_config for (company_id, customer_id) where effective_to IS NULL. If is_interstate = true: $igst_amount = (base+extra) * igst_rate / 100$. Else: $cgst = sgst = (base+extra) * cgst_rate / 100$. Both rates expressed as percentages, not fractions.

6. Module Specifications - Daily Work Panel

The Daily Work panel is where the operator spends most of their day. Duty slips are created here, aggregated into bills, edited or cancelled when mistakes happen, and printed for handing to the driver or sending to the customer. The four pages below - Duty Slip, Billing, Change/Cancel Bill, and Print Bill/Duty Slip - cover the complete daily operational cycle.

6.1 Duty Slip page

A two-section form: top half captures booking info (customer, vehicle, booking date, guest details, pickup/drop locations), bottom half captures duty info (start/end datetime, opening/closing km, extra charges, driver details). On save, the SPA calculates total_km and total_hours client-side, looks up the rate from master.rates for the selected customer/vehicle/duty_type triple, and posts the row to operations.duty_slips. The trigger operations.fn_assign_duty_slip_no() assigns the visible duty slip number from document_sequences.

6.2 Flexible duty type - custom rate popup

When the operator selects duty_type = flexible on the duty slip form, the SPA suppresses the standard rate lookup entirely. Instead, on save, a modal popup collects the **custom rate** amount and a free-text remark explaining the rate (e.g. "airport drop - flat Rs 1,500 negotiated over phone"). The popup also captures whether this rate is a one-off (stored on the duty slip only) or should be persisted as a new rate row in master.rates for future reuse. The duty slip is saved with rate_id = NULL and custom_rate populated; the bills trigger treats the custom_rate as the base_amount when aggregating into a bill.

6.3 Billing page

The billing screen lets the operator select a customer, see all unbilled duty slips for that customer (status = open or closed, bill_id IS NULL), tick the ones to include in a new bill, optionally adjust remarks, and click **Generate Bill**. The SPA posts to POST /rest/v1/rpc/generate_bill - a Postgres function that in a single transaction: (a) inserts a row into billing.bills with a trigger-assigned bill_no from document_sequences for key = "bill", (b) inserts rows into billing.bill_duty_slips for each selected duty slip, (c) updates duty_slips.bill_id and duty_slips.status = "billed", (d) fires the GST trigger to populate cgst/sgst/igst, and (e) inserts a "sale" entry into accounts.ledger_entries. The function returns the complete bill row.

Bill field	Source	Computation
bill_no	document_sequences	Prefix + zero-padded next_value + Suffix
base_amount	sum(duty_slips.base_amount)	SUM over linked slips
extra_amount	sum(extra*_amount + night_halt + driver_allowance + other)	SUM over linked slips
total_before_tax	base_amount + extra_amount	Computed column
cgst_amount, sgst_amount, igst_amount	gst_config	Trigger on bill insert
grand_total	total_after_tax + round_off	round_off set to make grand_total a whole rupee

Table 6.1 - Bill field computation map.

6.4 Change / Cancel Bill page

A bill is mutable until the customer receives a printed copy. After that, the operator should issue an **amended bill** (cancellation + reissue with new number) rather than editing the existing one. The Change/Cancel page supports three actions: (1) **Edit metadata** - change bill_date or remarks without touching duty slips; (2) **Add/Remove duty slip** - opens a popup listing unbilled duty slips for the same customer; the operator adds or removes one, the bill totals recompute via trigger; (3) **Cancel bill** - sets status = "cancelled", cancelled_at = now(), cancelled_by = current_user, frees the linked duty slips (bill_id = NULL, status = "closed"), and posts a reversal entry to accounts.ledger_entries. Cancellation is irreversible - the audit_log captures the before-state.

6.5 Print Bill or Duty Slip page

A print preview screen that renders the bill or duty slip as a PDF in the browser using @react-pdf/renderer. The operator picks a document type (Bill or Duty Slip), enters the number, the SPA fetches the row plus joined customer/vehicle/company info, and shows an A4 preview with a Print button. The same preview is reachable directly from the Billing page (a *Print* icon next to each bill row) and from the Duty Slip page. The PDF template is a single React component shared by both entry points so the layout is identical regardless of how the operator reaches it.

7. Module Specifications - Accounts Panel

The Accounts panel is intentionally minimal - it exposes a single page (the Ledger Book) and a single form (Record Receipt / Payment). All financial events in the system already write themselves to `accounts.ledger_entries` via triggers, so the Accounts panel is mostly a viewer with the ability to add manual entries. This keeps the panel easy to use for non-accountant operators while still giving the full general-ledger view to a trained accountant.

7.1 Ledger Book page

A scrollable ledger table filtered by customer (or all customers) and date range. Each row shows `entry_date`, `entry_type`, narration, debit (customer owes more), credit (customer paid), running balance, and linked bill/duty slip numbers as clickable links that jump to the corresponding page. The page also exposes a **Record Receipt** button (opens a small form: customer, amount, mode, reference, date) and a **Record Payment** button (vendor/driver payment out). Both insert rows directly into `accounts.ledger_entries` with the appropriate `entry_type`. Manual adjustments are logged with `entry_type = adjustment` and require a free-text narration of at least 20 characters.

7.2 Auto-posting rules

- **Bill issued** (status: draft -> issued): auto-insert a sale row with `debit = grand_total`, `narration = "Bill BILL-0001 raised against CUSTOMER"`
- **Bill cancelled**: auto-insert a sale row with `credit = grand_total`, `narration = "Reversal - Bill BILL-0001 cancelled"`
- **Duty slip closed** without bill: no ledger entry yet (the duty slip becomes billable; the bill triggers the sale entry)
- **Receipt recorded manually**: insert `entry_type = receipt`, `credit_amount = entered amount`, `linked_bill_id = NULL` (will be allocated to oldest outstanding bill on the customer reconciliation screen, future enhancement)

8. Module Specifications - Reports Panel

The Reports panel is read-only - every screen here is a saved SQL view rendered as a sortable, filterable table with a Print button. Three reports cover the daily operational needs: Bill Cover (summary list), Bill Register (detailed one-by-one view), and Duty Register (duty slip list). All three reuse the same React table component (TanStack Table v8) with different column configs and query functions.

8.1 Bill Cover report

A summary list of bills filtered by guest name, customer name, and date range. Each row shows: `bill_no`, `bill_date`, customer name, guest name (if any), duty slip count, base + extra, total tax, grand total, status. The report is the operator's at-a-glance view of what was billed in a period. A Print button exports the filtered set to PDF as a tabular report (no individual bill PDFs). The underlying query is a single `SELECT` against

billing.bills joined to master.customers and aggregated by bill_id.

8.2 Bill Register report

A detailed walk through every bill, one at a time, with a left-side navigation list and a right-side detail panel showing the full bill: company header, customer block, line-item duty slips with km/hours/amounts, GST breakdown, grand total, and a Print Bill button. The list supports filter by date range, customer, and status. This screen is what the accountant uses to walk through bills when reconciling - clicking a bill in the left list immediately loads its full preview on the right.

8.3 Duty Register report

A summary of every duty slip in a date range, sortable by date, customer, vehicle, or status. Columns: duty_slip_no, booking_date, customer name, vehicle registration, duty_type, total_km, total_hours, total_amount, bill_no (if billed), status. Useful for vehicle utilization analysis and unbilled duty slip identification. A Print button exports the filtered set to PDF as a tabular report.

9. Cross-Cutting Concerns

Authentication, audit logging, and PDF generation cut across all four panels. This chapter consolidates the design for each so the panel-specific chapters can refer back here instead of repeating the mechanism.

9.1 Authentication and role model

Supabase Auth (GoTrue) handles credentials. Four roles are defined in the user_role enum: **owner** (full access, manages users), **operator** (creates duty slips and bills), **accountant** (records ledger entries, views reports), **viewer** (read-only access to reports only). The role is stored on core.user_profiles.role and propagated into the JWT as the user_role claim via a Supabase Auth hook on token issuance. The SPA reads the claim from the access token and uses it both for UI gating (hiding buttons the user cannot use) and for the API call headers (PostgREST enforces RLS server-side independently, so a tampered UI cannot bypass security).

9.2 Audit log mechanism

A single generic trigger function system.fn_audit_row() attaches to every business table (customers, vehicles, rates, gst_config, duty_slips, bills, ledger_entries, document_sequences). On INSERT/UPDATE/DELETE, it inserts a row into system.audit_log capturing the table name, record id, action, old row as JSONB, new row as JSONB, and the current user id from auth.uid(). The audit log is append-only - no UPDATE or DELETE triggers are attached to audit_log itself, and an RLS policy denies all writes except via the trigger function. The audit log table is partitioned monthly by changed_at to keep query performance reasonable as the table grows.

9.3 PDF export pipeline

All printable documents - bills, duty slips, Bill Cover report, Duty Register - are rendered in the browser using @react-pdf/renderer. This library produces vector PDFs without a server round-trip, keeping the Oracle VM free of headless Chrome. Templates are React components under src/templates/pdf/: BillPDF.tsx,

DutySlipPDF.tsx, BillCoverReportPDF.tsx, DutyRegisterReportPDF.tsx. Each takes a typed data prop (fetched via TanStack Query before opening the print preview). The rendered PDF opens in a new browser tab via pdf.blobURL(), where the operator can review and use the native browser print dialog to send to a physical printer or save as a file. Company logo, address, and GSTIN are pulled from core.companies at render time so a company-info change auto-reflects in subsequent prints.

Why client-side PDF

Server-side PDF rendering with Puppeteer / headless Chrome would add ~300 MB RAM and a separate container, plus a per-print cold-start delay. @react-pdf/renderer is pure-JS, has no native dependencies, and produces vector output identical across browsers.

10. Deployment & Free Tier Strategy

The full stack runs on a single Oracle Cloud Always Free ARM A1.Flex instance. The free tier provides up to 4 OCPUs and 24 GB of RAM shared across up to 4 instances, plus 200 GB of block storage. The taxi ERP needs only one of those instances - 2 OCPUs and 8 GB RAM is plenty for the application workload, leaving headroom for the monitoring sidecar and future expansion. This chapter walks through the deployment topology and the backup strategy.

10.1 VM provisioning checklist

- Compute shape: **VM.Standard.A1.Flex**, 2 OCPU, 8 GB RAM, Ubuntu 22.04 image (canonical-Ubuntu-22.04-aarch64)
- Block storage: 50 GB boot volume (default) - sufficient; attach a second 100 GB block volume for Postgres data if growing
- SSH key: ed25519, no passphrase (CI/CD will use this key)
- Ingress rules: **none** - Cloudflare Tunnel handles all ingress. Leave all inbound ports blocked. Egress: allow 443 (Cloudflare Tunnel), 22 (Backblaze B2 via S3 API), 53 DNS
- Swap: configure 4 GB swap file (Oracle A1 instances have no swap by default, Postgres can OOM-kill on memory pressure)

10.2 Docker Compose topology

Container	Image	Purpose	RAM
kong	kong:3.4	API gateway on :8000	256 MB
postgres	postgres:16-alpine	Main DB on :5432	2 GB
rest	postgrest/postgrest:v12.0.2	Auto-CRUD REST	128 MB
auth	supabase/gotrue:v2.158.0	Auth service on :9999	256 MB

Container	Image	Purpose	RAM
storage	supabase/storage-api:v1.11.0	S3-compatible on :5000	256 MB
meta	supabase/postgres-meta:v0.83.2	Schema admin UI on :8080	128 MB
studio	supabase/studio:2024.06	Admin dashboard on :3000	256 MB
uptime-kuma	louislam/uptime-kuma:1	Monitoring on :3001	256 MB
cloudflared	cloudflare/cloudflared:latest	Tunnel client	64 MB
Total estimated			~4.1 GB

Table 10.1 - Docker Compose topology. Sits within 8 GB VM headroom.

10.3 Backup strategy

A nightly `pg_dump` in custom format (-Fc) is taken at 02:00 by a `pg_cron` job. The dump is encrypted with age (a modern, simple file-encryption tool) using a public key whose private half is stored off-VM (operator's laptop). The encrypted dump is pushed to Backblaze B2 via `rclone` into a versioned bucket; 30 days of nightly dumps are retained, plus a monthly snapshot retained for 12 months. Total storage requirement is well under the 10 GB B2 free tier for a single-taxi-business DB. Restoring is a documented runbook: pull the dump from B2, decrypt with the private key, `pg_restore` into a fresh Postgres container on any VM.

11. Modular Extensibility Plan

The architecture is deliberately modular so that a future panel can be added without touching existing ones, and an unwanted panel can be removed without breaking the others. Three principles make this work: schema namespaces per panel, dependency direction rules, and soft references instead of hard foreign keys for optional cross-panel links.

11.1 Schema namespaces per panel

Each panel owns a Postgres schema: `master.*`, `operations.*`, `billing.*`, `accounts.*`, `reports.*` (views only). Removing a panel = `DROP SCHEMA billing CASCADE` in reverse dependency order, which cleanly removes its tables without affecting others. Adding a panel = `CREATE SCHEMA payroll` with its own tables; the new panel's SPA routes register lazily via a feature-flag check.

11.2 Dependency direction

master is referenced by **operations**, **billing**, **accounts**. **operations** is referenced by **billing**, **accounts**. **billing** is referenced by **accounts**. **accounts** references nothing above it. **reports** is read-only views across all. This strict layering means a lower panel can be removed without breaking an upper panel (e.g. remove **billing** - duty slips still work, just cannot be aggregated). Adding a panel in a new layer below **accounts** is safe; adding one

between two existing layers requires migration of dependencies.

11.3 Soft references for optional cross-panel links

When a panel-optional link is needed (e.g. a future "Driver Management" panel that links drivers to duty slips), use a nullable bigint `driver_id` column without a FOREIGN KEY constraint. Validation moves to the application layer: the SPA checks the driver exists before save. This decouples the panels so the Driver panel can be removed without breaking `duty_slips` (the column stays NULL). Hard FKs are reserved for mandatory links (`customer_id`, `vehicle_id`, `company_id`) where a missing parent would corrupt the data.

Removal runbook

To remove the Reports panel: drop the reports schema (views only - safe CASCADE), remove the Reports routes from the SPA router, remove the Reports entry from the navigation sidebar. No data loss. To remove the Billing panel: first cancel all open bills, set `duty_slips.bill_id = NULL` for all rows, drop the billing schema, remove the SPA routes. Duty slips remain queryable but unbilled.

12. Implementation Roadmap

A phased plan that gets a minimum-viable product in front of the operator within two weeks and reaches full feature parity with the specification within eight weeks. Each phase ends with a demonstrable milestone so progress is visible. Risk register at the end of this chapter lists the things most likely to slip the timeline.

Phase	Scope	Duration	Milestone
1. Foundation	VM provisioning, Docker Compose, Supabase self-host, Auth, RLS, audit triggers	1 week	Login works, can create a company
2. Master Panel	Company Detail, Utilities (Vehicles + Doc No. Control), Customer, Rate Management, GST Management	2 weeks	All reference data entered
3. Daily Work - Duty Slips	Duty Slip CRUD incl. flexible duty popup, rate lookup, list & search	1.5 weeks	Can create and print a duty slip
4. Daily Work - Billing	Bill generation from duty slips, GST trigger, change/cancel bill, print bill PDF	1.5 weeks	End-to-end bill from duty slip
5. Accounts Panel	Ledger Book, manual receipt/payment entry, auto-posting triggers	1 week	Ledger reflects bill activity
6. Reports Panel	Bill Cover, Bill Register, Duty Register, PDF export of each	1 week	All three reports printable
7. Hardening	Backups, Uptime Kuma, Cloudflare Tunnel, rate limiting, smoke tests	0.5 week	Production-ready

Table 12.1 - Implementation phases. Total: ~8.5 weeks of solo effort.

12.1 Risk register

Risk	Likelihood	Mitigation
Oracle reclaims Always Free instance for inactivity	Medium	Uptime Kuma pings every 60s; never idle
Supabase self-hosted upgrade breaks schema	Low	Pin image versions in Compose; test on staging VM first
Cloudflare Tunnel egress blocked by ISP	Low	Cloudflare Warp as fallback; or ngrok as backup
Backblaze B2 free tier exhausted by audit log growth	Low	Partition audit_log monthly, archive old partitions quarterly
Postgres OOM on large bill aggregation	Medium	Cap bill_duty_slips per bill at 100; paginate Reports queries
Client-side PDF rendering slow on low-end devices	Medium	Lazy-load @react-pdf/renderer; show progress spinner
GST rate changes mid-fiscal-year	High	gst_config effective_from/to handles retroactive bills correctly

Table 12.2 - Risk register. Reviewed at the end of each phase.

The roadmap is conservative - a focused solo developer can compress phases 5 and 6 into a single week, but the buffer accommodates the inevitable edge cases in GST calculation, bill renumbering, and printable PDF layout that tend to consume the last 20% of effort. After phase 7, the system enters maintenance mode: monthly security patch application (automated via Watchtower on the Docker side), quarterly review of audit_log growth, and annual review of the Oracle Free Tier policy in case Oracle tightens quotas and a paid migration becomes necessary.